

Tutorial BGP Lab

This lab studies Border Gateway Protocol route withdrawal and failover across four autonomous systems. You will run INET's BGPv4 implementation, stop and restart one router, watch the forwarding path in Qtenv, and analyze ping loss and round-trip time. The supplied BgpFailoverLab folder contains its topology, address plan, BGP configuration, and omnetpp.ini.

1 Prepare the workspace

Use OMNeT++ 6.4.0 with INET 4.7.0 or compatible versions. INET must be present, built, and visible in the same IDE workspace. The BGP lab uses INET Router and StandardHost modules, BGPv4 and TCP, PingApp, ScenarioManager, and canvas visualizers. Keep the NED, both XML files, and omnetpp.ini together.

1. Start the OMNeT++ IDE and choose a workspace. If INET is not installed, install it and build the inet project. Confirm the project is open.

2. Import BgpFailoverLab with File > Import > General > Existing Projects into Workspace. The project descriptor already declares inet as a reference.

3. Confirm that BgpFailoverNetwork.ned, IPv4Config.xml, BGPCconfig.xml, omnetpp.ini, and README.md sit at the project root. Both XML files must be found from the run working directory.

If you create a new project, copy all files to its root, mark the root as a NED source folder, add inet as a project reference, and point the launch working directory to that root.

4. Verify the inet reference, NED source folder, and run configuration. Build INET before launching this lab. No custom BGP C++ compilation is required because INET supplies the implementation.

5. Check that BgpFailoverNetwork resolves without NED errors and that the XML files are readable from the launch working directory.

2 Import an existing OMNeT++ project

The BgpFailoverLab folder already has a .project file. Use Existing Projects into Workspace rather than creating a nested project. Verify its inet reference, NED folder, and run working directory. If the IDE cannot launch it, create an OMNeT++ Simulation launch for omnetpp.ini and load the INET library and NED path.

3 Understand what the files build

File or group	Role in the simulation
BgpFailoverNetwork.ned	Defines four BGP routers, two hosts, fast and backup links, ScenarioManager, and canvas labels.
omnetpp.ini	Sets BGP/TCP timers, PingApp traffic, router shutdown/restart, route visualization, and recording.
BGPConfig.xml	Assigns AS numbers, originated networks, and four external BGP sessions.
IPv4Config.xml	Assigns router/host interface addresses and default routes to the two end hosts.
README.md, .project	Explain the lab and link the project to the existing inet workspace project.

Four BGP routers form a diamond between a red source and green destination: west is AS 65001, north is AS 65002, east is AS 65003, and south is AS 65004. The upper west-north-east path uses 50 us links; the lower west-south-east path uses 300 us links. BGPConfig.xml originates the edge networks and declares four external sessions. Link colors show topology; BGP decides forwarding.

4 Run BGP

1. Open omnetpp.ini and run the General configuration with Qtenv. Select BgpFailoverNetwork if the launch asks for a network. The simulation limit is 160 s.
2. Use BgpFailoverLab as the working directory so BGPConfig.xml and IPv4Config.xml resolve. Include INET as a project reference or load its library and NED folder.
3. Watch the black forward-ping route overlay, red DROP labels, and source PingApp output. At 60 s ScenarioManager shuts down north; at 100 s it starts north again. The lower path remains available while north is down.

Parameter guide. sim-time-limit = 160 s covers startup, outage, and recovery; seed-set = 41 makes the run repeatable. BGP startDelay = 2 s delays session startup, connectRetryTime = 5 s controls retry attempts, holdTime = 6 s limits silence before a session is considered dead, and keepAliveTime = 2 s sends liveness messages. PingApp starts at 20 s, sends a 256 B request every 1 s, and targets 10.0.3.100. TCP uses MSS 1024 B and advertisedWindow 14336 B. ScenarioManager shutdown at 60 s and startup at 100 s define the failure experiment. Changing timers changes convergence and loss, while changing link delays changes RTT without changing the selected topology.

What the protocol modules measure

INETs BGPv4 modules exchange OPEN, KEEPALIVE, UPDATE, and withdrawal information over TCP. BGPConfig.xml supplies AS numbers, advertised networks, and session endpoints. IPv4Config.xml assigns interface addresses and end-host default routes; it does not install static inter-AS routes. PingApp records sent requests, lost replies, and RTT. The route overlay follows forwarded echo requests, while BGP state can be inspected in routing tables and the event log.

Recording guide. scalar-recording is enabled for summary values, while vector-recording is disabled globally to keep files small. The source RTT vector is explicitly enabled. Enable additional BGP, TCP, or routing-table vectors only for a focused experiment because session keepalives and updates can produce long traces. Always record the timer, link-delay, traffic, and failure parameters beside results.

5 Observe the route change

Simulation time	What to inspect
0-20 s	BGP/TCP sessions start and exchange route information. PingApp has not begun sending.
20-60 s	Pings use the initially preferred upper path. Note the black route overlay and baseline RTT.
60-100 s	ScenarioManager stops north. Watch withdrawal, possible lost replies, and pings on the slower lower path.
100-140 s	North restarts. Watch BGP session recovery and the route return to the upper path.
140-160 s	Confirm the stabilized path and let the run finish to write scalars and the RTT vector.

BGP learns reachability through the configured external sessions. North shutdown removes the upper AS path; an UPDATE or withdrawal and routing-table change allow the lower path to carry pings. After north restarts, the upper path can return. Use the route overlay, routing tables, event log, and RTT together. A visible link color is a topology cue, not proof of active forwarding. Qtenv animation speed does not change simulation time.

6 Find and analyze results

1. Let the run reach 160 s and refresh the results folder. The .sca file contains scalar statistics, the .vec file contains the enabled source RTT vector, and the .elog file helps locate control and path changes.
2. In Browse Data, filter on BgpFailoverNetwork.source.app[0]. Inspect Pings sent, ping loss rate, RTT statistics, and the PingApp console summary. Also inspect BGP module scalars and router routing tables for learned prefixes.
3. Plot source.app[0].rtt against simulation time. Compare samples before 60 s, while north is down, and after its restart at 100 s. A missing RTT sample means no reply was measured for that request, not zero latency.
4. Use the forward route overlay or event log to identify the next hop at west in each interval. Compare loss and RTT, and distinguish the shutdown event from BGP's subsequent routing response. For a broader study, vary hold timers, link delays, path properties, and seeds deliberately.

Recorded result	How to interpret it
Pings sent	Number of requests PingApp issued after its 20 s start.
ping loss rate (%)	Percentage of requests for which source PingApp received no echo reply.
PingApp received replies	Count shown by PingApp run summary; it can be less than Pings sent.
RTT minimum/mean/maximum (s)	Fastest, average, and slowest successful source-to-destination-to-source exchanges.
source.app[0].rtt vector	Time series of successful reply delays; gaps do not represent zero latency.
BGP timers	startDelay, keepAliveTime, holdTime, and connectRetryTime shape session startup and failure detection.
link and traffic parameters	Link delays, packet size, send interval, run length, and failure schedule shape RTT, loss, and exposure.
BGP module statistics	Use per-router scalars with routing tables and the event log to explain convergence.

The source ping loss count represents missing echo replies. It does not separately count requests that reached the destination but lost their replies. The reference 160 s run sent 141 pings and received 140 replies, with about 0.598 ms on the preferred route and 1.598 ms on the backup route. Reproduce values after any parameter change rather than assuming them.

7 Student analysis questions

- Map west, north, south, and east to their AS numbers. Which prefixes are originated at the two edge autonomous systems?
- Find the selected route before 60 s. Which route overlay, routing-table entry, or event-log message supports that selection beyond the static link colors?
- Change holdTime and keepAliveTime separately. Predict how quickly a failed session is detected and how many pings may be affected.
- Change the upper and lower link delays or the PingApp sendInterval. Explain the expected effect on RTT, loss, control traffic, and vector size.
- Report Pings sent, replies received, loss rate, RTT minimum, mean, and maximum. Record sim-time-limit, seed-set, traffic timing, BGP timers, and failure times beside the results.

8 Troubleshooting and limits

- Unknown INET NED type or missing BGP module: confirm inet is open, built, referenced, and on the run NED and library paths.
- XML file not found: run from BgpFailoverLab, where BGPCfg.xml and IPv4Cfg.xml sit beside omnetpp.ini. Check the launch working directory.
- No ping delivery before 20 s: this is expected because PingApp starts at 20 s. If delivery never begins, inspect BGP sessions, addresses, and the INET reference.
- No route switch at 60 s: confirm ScenarioManager runs the script and north has hasStatus = true. Check the event log and routing tables after shutdown.
- No black route overlay or red labels: use QtEnv, enable the canvas visualizer, and pause near traffic or loss events. Blue and gold lines depict physical connections.
- This four-AS model uses fixed link delays and a scheduled failure. It omits commercial policies and Internet-scale behavior. Loss and convergence times depend on timers, traffic, and this topology.

Reference documentation

OMNeT++ User Guide: <https://doc.omnetpp.org/omnetpp/UserGuide.pdf>

INET installation: <https://inet.omnetpp.org/Installation.html>